

Laya：本地开源的"系统一"决策模型

—— 不写一个字的 AI，怎么在 33 毫秒内替你做判断？（附 Jev vs Laya 本地复现对比实验）

Apache-2.0 开源

421M 参数

单次前向

100+ 语言

`pip install laya`

拾遗任务 1a45286b · 源帖：小墨同学 X 帖《Jev 和开源本地大模型到底哪个更强？我直接在本地部署了 Laya 模型做系统性测试》· 学习产出 2026-10-03 · 全部实验在本机（Windows 11 + RTX 4070，CPU 推理路线）真实跑通

目录：小白工程师的 12 个疑问

1. Laya 是什么？和 Jev 什么关系？
2. 不生成文字，怎么"做决策"？——非自回归单次前向
3. 421M 参数里装的是什么骨架？
4. Choice / Score / Noul 三原语在代码里长什么样？
5. RLCD 训练：为什么"诚实报概率"对模型最划算？
6. 概率校准是什么？为什么会"自信地错"？
7. Router：英文模型离了英语为什么崩得比乱猜还惨？
8. 置信度门控：不确定时模型会"举手说不会"吗？
9. 推主说 Jev 准确率完胜，仓库却说 Laya 更高——评测口径之谜
10. 动手实测：24 题双语分诊基准，Laya vs 云端大模型（本机复现）
11. 部署与生态：pip / ONNX / MLX / laya.cpp / MCP 一览
12. 什么时候用 Laya，什么时候用 LLM？

1 Laya 是什么？和 Jev 什么关系？

一句话：**Laya 是 Jev 的开源平替**——一个不生成任何文字、只对输入做"判断"的本地决策模型。你给它一段工单/消息 + 几个问题，它一次前向直接吐出带校准概率的结构化决策。

故事线很有戏剧性：作者 Nandakishor M 早在 2025 年就发表过非自回归决策模型的论文

(arXiv:2503.23303、2510.01237)；2026 年 9 月 TypeSafe AI 发布闭源决策模型 **Jev**（我们已学过：[[SystemOne 决策模型 Jev]]，Choice/Score/Noul 三原语，\$21/百万次决策），作者认为"这是我先做的方向"——于是把自己的系统**完整开源**出来，取名 Laya：2026-09-18 上架 GitHub，一个月拿下 **30,000+ 星**。

关系一句话：Jev = 闭源托管 API（不能本地跑、数据要出域）；Laya = 同一套三原语接口的**开源自托管实现**（Apache-2.0，权重在 HuggingFace，pip install 就能本地跑）。小墨同学的帖子干的事就是：把两者放在一起实测。



来源：repo README / 作者 dev.to 文章（注：评论区有人指出该文章发布当天注册账号，宣传性内容需打折听，结论以仓库实测为准——这正是我们做复现实验的原因）

2 不生成文字，怎么"做决策"？——非自回归单次前向

大模型回答"该派给账务组吗"要一个 token 一个 token 地往外蹦（自回归生成），每个字都在付解码的钱，还可能蹦出半个 JSON。Laya 完全反着来：**输入整句喂进去，一次前向传播 (forward pass)，所有答案的概率分布同时出来**——没有解码循环，所以延迟固定、结构天然合法、更不会"幻觉"出一个不存在的选项。

自回归 LLM (生成式)

"该派给" → "账" → "务" → "组"

逐 token 解码，越写越贵

延迟随输出长度变

JSON 可能蹦坏，需要修复

典型一次：3~15 秒

VS

Laya (非自回归)

整句输入 → 编码器一次前向

所有问题的概率分布同时输出

延迟固定（一次矩阵乘法的量级）

结构天生合法（就是 schema 上的分布）

GPU 33ms / 本机 CPU ~250ms

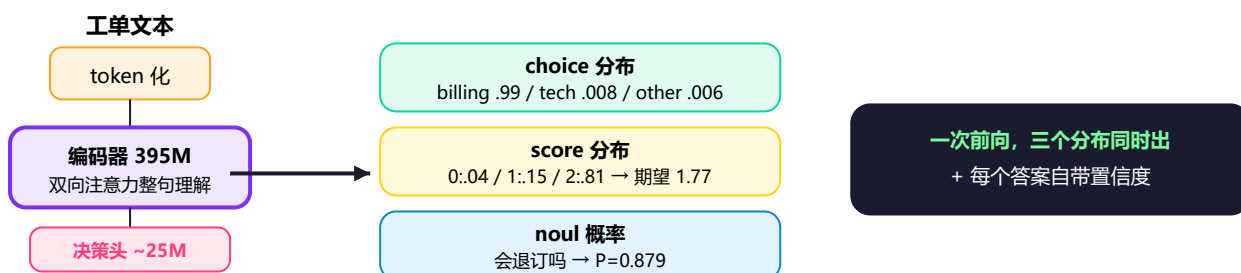
类比落回术语：就像**阅卷 vs 写作文**——LLM 在写作文（要一个字一个字写出来）；Laya 在做选择题（卷子看一眼，把答题卡涂了）。选择题当然快，而且答案不可能"跑题"。代价是它**只能答题，不能作文**——你不能让 Laya 帮你写退款邮件。

3 421M 参数里装的是什么骨架？

Laya 不是一个模型，是一组检查点（checkpoint）+ 一个路由器。每个 checkpoint 都是"双向编码器 + 决策头"：

checkpoint	骨架	参数量	上下文	管什么
laya (english)	ModernBERT-large	421M (395M 编码器 + ~25M 决策头)	512 tokens	英文最准
laya-multilingual	mmBERT-base	322M	1024 tokens	100+ 语言
laya-typed-decisions	ModernBERT-large 微调	421M	1024 tokens	四类工作流微调版，官方基准 0.362→ 0.766

ModernBERT / mmBERT 是什么：BERT 家族的现代改进版——**双向注意力编码器**（看整句而不是从左往右看），专为"理解 + 分类"设计，不做生成。mmBERT 在此基础上用多语言语料训练，所以 322M 就能覆盖 100+ 语言。**小模型的"判断力"主要来自训练目标，不是参数堆量。**



4 Choice / Score / Noul 三原语在代码里长什么样？

跟 Jev 的三原语一一对应（接口设计故意兼容）。在 Laya 里它们就是 **schema 描述**：你声明问题类型和选项，前向一次全部作答：

```
questions = {
  "department": {"type": "choice",    # 从预设选项挑一个
    "criteria": {"billing": "发票/付款/退款", "technical": "故障/报错", "other": "其他"}},
  "urgency":    {"type": "score",     # 0~2 连续量表
    "criteria": ["不紧急", "尽快", "业务停摆"]},
  "churn_risk": {"type": "noul",      # 校准的真假概率（no/ul = 是非判断）
    "instructions": "用户是否威胁退订？"},
}
r = router.predict(工单文本, questions)  # 一次调用三题全答
```

实测返回（本机 lab01 真实输出，billing 工单）：

```
department → choice=billing    概率 {billing: 0.9865, technical: 0.008, other: 0.0055}  conf 0.93
urgency     → score =1.77       概率 {0: .039, 1: .15, 2: .811} ("blocking" 2 档最可能)
churn_risk  → noul = 0.879      (87.9% 概率这人有退订风险)
```

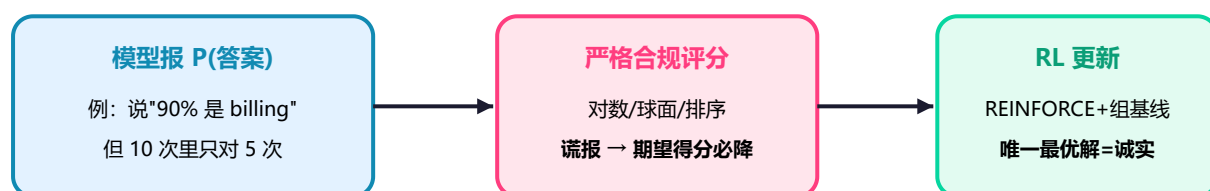
注意 score 的妙处：它不是分类挑档，而是输出**整个量表的分布再取期望**（1.77 = 介于"尽快"和"停摆"之间偏严重）——比硬分类保留更多信息，可以做"严重度排序"。

5 RLCD 训练：为什么"诚实报概率"对模型最划算？

Laya 的准确率不是标注+交叉熵喂出来的，而是**强化学习**：RLCD (Reinforcement Learning for Calibrated Decisions, 面向校准决策的强化学习)。核心机关在奖励函数——用的是**"严格合规评分规则" (strictly proper scoring rules)**：对数评分、球面评分、排序概率评分的组合。

严格合规评分规则 = 撒谎不划算的打分表：数学上可证明，只有当你报出的概率等于你真实的信念时，期望得分才最大。报高了对不上会罚，报低了也罚。**所以模型被"逼着"学会诚实**——这就是它置信度可信的来源（而普通分类训练没有这个性质，容易训练出"永远 99% 自信"的模型）。

工程实现：REINFORCE + GRPO 式组基线（8 个噪声采样一组内互相比），多轮任务加 $TD(\lambda=1)$ 。另有一个 **Act/Escalate 头**：置信度低于约 62.5% 时学会"升级给人"，不是硬答。

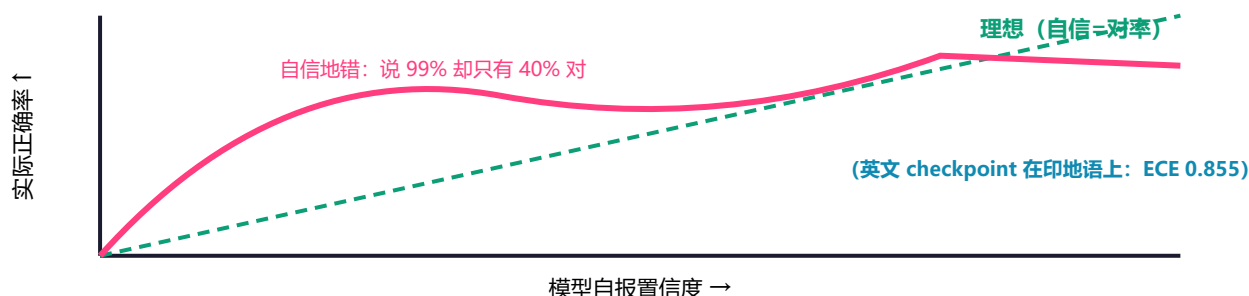


6 概率校准是什么？为什么会"自信地错"？

校准 (calibration)：模型说"90% 把握"的事里，真实发生率就该接近 90%。衡量指标 **ECE**（期望校准误差，越低越好）：把预测按置信度分桶，比一比"平均自信"和"实际对率"的差距。

修校准的标准手术是**温度缩放 (temperature scaling)**：对输出的 logits 除以一个温度 T 再 softmax—— $T > 1$ 让分布变平（不那么自信）， $T < 1$ 变尖。Laya 每个 checkpoint 按选项数分桶存了一套 T 。

我们撞上的真实案例：本机首次调用就打印 RuntimeWarning——`choice:11+` 桶的温度 `0.1006` 超出 `[0.5, 5]` 合法区间，被夹到 `0.5`，该桶置信度视为未校准。仓库 BENCHMARKS.md 用一整节交代这事的修复史（issue #42、#208）：温度夹紧让 51 语言宏平均 ECE 从 0.733 降到 0.571，而**准确率一个字没变**（温度缩放不改变 argmax）——这就是"校准"和"准确"是两回事的活教材。



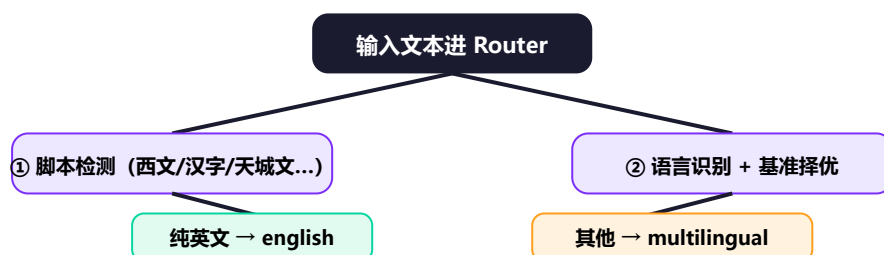
7 Router: 英文模型离了英语为什么崩得比乱猜还惨?

三个 checkpoint 各有所长, Router 自动挑: 脚本检测 (拉丁/西里尔/汉字/天城文...) 是第一信号, 再看语言。为什么这么较真? 看仓库自己晒的伤疤:

英文 checkpoint 的非英语崩溃: 20 选项意图分类上, 印地语准确率 **0.100**, 比随机乱猜 (0.050) 高得有限, 但置信度照样拉满 (ECE 0.855)。文档原话: "it does not gently degrade off English, it collapses" (不是平缓变差, 是塌方)。所以路由错了不是"差一点", 是"自信地全错"。

场景	english (421M)	multilingual (322M)
MASSIVE 意图 · 英语	0.783	0.657
MASSIVE 意图 · 非英语	0.306	0.451
XNLI · 非英语	0.521	0.731 (+21 分)

本机实测 (lab01): 中文工单 我三月被扣了两次款... 自动路由到 multilingual, 答案 billing、概率 0.9955——路由机制真实工作。这也顺手纠正了推文一个口径: **"Laya 只支持英文"不准确——是英文 checkpoint 只撑英文, 路由会换模型** (推主 9-19 测试时可能只用了 english checkpoint 或早期版本)。



8 置信度门控: 不确定时模型会"举手说不会"吗?

会, 而且是设计出来的: 结果里每个答案都带 `action.act_probability` (该不该自动执行) 和 `confidence`。你可以设 `min_confidence` 门槛——低于门槛的不是硬答, 而是弃权 (abstain) 升级给人, 且官方实现是 fail-closed (没把握宁可不做)。

官方口径: 校准后 top-50% 最有把握的预测准确率 92.2% (全量是 65~84%) ——**"挑有把握的答"能把准确率买回来, 代价是覆盖率**。这就是决策模型接进生产线的安全阀。

实测泼冷水: 我们 24 题基准里, 按 `conf ≥ 0.8` 过滤后只剩 11 题可答, 子集准确率 72.7%——**并没有比全量 75% 更高**。教训: 置信度门控的前提是校准真的好; 某个 checkpoint 校准差时, 门控只是随机少答几题。**上生产前必须拿自己的数据重新校准 (repo 自带 `fit_abstention_thresholds` / 温度拟合工具)**。

9 推主说 Jev 准确率完胜，仓库却说 Laya 更高——评测口径之谜

三方数字摆一起，看起来打架，其实各说各话：

来源	测的是什么	结果
小墨同学 (X 帖)	自建 100 个真实事件场景，Laya (仅英文) vs Jev	Jev 判定率 100% vs Laya ~57% (双方都全通过)；速度 Laya 完胜
Laya 仓库 BENCHMARKS	公开基准 typed-decisions 2000 决策	微调 Laya 0.766 vs Jev 公开值 0.727 (Jev 数字是第三方发表值，未实测)
作者 dev.to 文	自报：23,024 题域内 83.8% / 2,400 题零样本 65.1%	注意：发布当天注册的账号，自宣性质
本次复现 (我们)	自建 24 题双语工单分诊，零样本	Laya 本地 dept 75% / churn 92% (见 Q10)

评测素养三条：① 自建测试集的"100%"只在同一张考卷上成立——推主的 57% 和仓库的 0.766 不是同一张卷子，不可直接比；② 零样本 vs 微调差一倍 (0.362→0.766)，问清"哪个 checkpoint"再谈准确率；③ 商家对比数字 (快 193 倍/便宜 444 倍/Jev 更准) 一律当"厂商自宣"，找独立复现——这正是我们 Q10 亲手干的事。

10 动手实测：24 题双语分诊基准，Laya vs 云端大模型（本机复现）

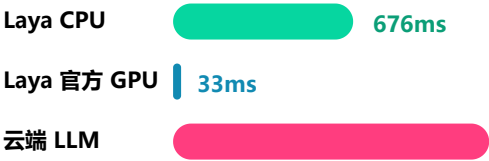
复现推主的评测范式（本地部署 + 同一张考卷比准确率和速度），但全部在自己机器上、数据自己造、每一步可复制。考卷：**24 道客服工单（12 英 + 12 中）**，**每题三问**——部门 choice / 紧急度 score / 退订风险 noul，人工标准答案。

维度	Laya 本地 (CPU)	云端大模型 (claude CLI / kimi)
dept 准确率	75.0%	95.8%
urgency 分档	54.2%	83.3%
churn 真假	91.7%	100%
三问全对	37.5%	79.2%
单题延迟 (中位)	676ms	6,693ms

三个**比分数更值钱**的发现：

- ① **决策模型也吃提示词工程**：12 道中文题的紧急度全线饱和在"2 级"（连"不着急"都判成停摆级）——因为 criteria 用英文写的、输入是中文，措辞错配。把 criteria 换成中文后：urgency 4/12 → **7/12**，dept 8/12 → **10/12**，且不再饱和。回填到全量 24 题：urgency 54.2% → **66.7% (9/12 英文 + 7/12 中文换中文措辞)**。schema 里的 criteria 就是决策模型的 prompt，语言要跟着输入走。
- ② **置信度门控不白给**：conf≥0.8 只剩 11 题，子集准确率反而 72.7%（低于全量 75%）——checkpoint 校准不达标时门控无效（呼应 Q8）。
- ③ **快是真快，但首选台要挑**：首调用 10.3 秒（下载+加载 checkpoint），暖态 CPU 中位 676ms；官方 GPU 33ms。速度对比云端大模型约 约 10 倍。

单题延迟对比（对数尺，越小越快）



首调用 10.3s = checkpoint 下载+加载（一次性成本，Router(preload=True) 可预热)

完整逐步操作见下方"动手实验"节；数据：exercise/laya_results.json、llm_results.json、zh_criteria 对照输出

11 部署与生态: pip / ONNX / MLX / laya.cpp / MCP 一览

路线	装法	适合
官方 Python	<code>pip install laya</code>	本教程路线, CPU/GPU/MPS 全吃
laya-ts (ONNX)	<code>npm install laya-ts</code> / reception/laya	Node.js / 浏览器 / Windows 无 Python 场景
laya-mlx (6.7k★)	Apple MLX 原生	Mac M 系列, 7~14ms (推主测试的路线)
laya.cpp / GGUF	mys/laya-GGUF 等	C++/CUDA/Vulkan, llama.cpp 系工具链
laya-server / Docker	官方 compose	团队自托管 HTTP 服务 (兼容 TypeSafe API 形状)
MCP / 框架集成	<code>laya[mcp]</code> <code>laya[langchain]</code> 等	给 Agent 当裁判工具

和 jev-router 接上: 我们学过的 jev-router ([[逐轮模型路由与哨兵代理]]) 靠 TypeSafe 云端做逐轮选档; 换成 Laya 后整条路由**可以完全本地**——隐私不出机器, 延迟还低两个数量级。生态里已有 oh-my-laya 等项目干这事。

12 什么时候用 Laya, 什么时候用 LLM?

用 Laya (决策模型)	用 LLM (生成式)
高频、低价值密度、延迟敏感的判断: 路由/分诊/审核/护栏	需要生成内容: 写邮件、写代码、总结
选项封闭、可枚举 (choice/score/是非)	答案开放、没法枚举
数据不能出机器 (Laya 全本地)	能接受出域或已自托管
要概率和置信度参与业务规则 (门控/弃权)	只要最终文本

本机 24 题复现给出的答案介于推主和仓库之间: **"Jev 完胜准确率"在零样本英文上部分成立, 但中文要换 checkpoint + 本地化 criteria; 速度本地完胜无争议。** 最稳的姿态: Laya 当毫秒级第一道闸, 没把握的升级给 LLM 或人——和 jev-router 的 fail-open 分层是同一个哲学。

✦ 动手实验：本地部署 + 对比评测（全部命令实测于 Windows 11 + RTX 4070, CPU 路线）

第 1 步：装环境（隔离 venv，不污染全局 Python）

怎么操作：

```
python -m venv F:\reminder\laya-decision-model\.venv
F:\reminder\laya-decision-model\.venv\Scripts\python.exe -m pip install laya
```

目的：装上 laya 0.3.24 (Apache-2.0, 自带 torch/transformers 依赖)。**实现了什么：**项目专属 venv + 依赖隔离。

✓ **预期结果：** `Successfully installed laya-0.3.24 ...` ; `.venv\Scripts\python.exe -c "import laya"` 无报错。

第 2 步：冒烟测试——三原语各来一发 (lab01_quickstart.py)

怎么操作：

```
cd F:\reminder\laya-decision-model
set PYTHONUTF8=1 && set HF_HOME=.hfcache
.venv\Scripts\python.exe exercise\lab01_quickstart.py
```

目的：验证安装 + 亲眼看到 choice/score/noul 的返回形状、中文自动路由。**实现了什么：**英文 billing 工单三问全中 (0.99/1.77/0.88)，中文工单路由 multilingual 也中 (0.9955)，技术故障工单换成 technical (0.92)。

✓ **预期结果：**打印 `routing=english` 与 `routing=multilingual`，choice 概率最高项与人工直觉一致；首次调用含 5~50 秒下载属正常。

第 3 步：自建 24 题双语考卷跑 Laya (lab02_laya_benchmark.py)

怎么操作：

```
.venv\Scripts\python.exe exercise\lab02_laya_benchmark.py
```

目的：复现“本地部署+系统性测试”——同一张考卷量准确率和延迟。**实现了什么：**dept 75% / urgency 54% / churn 92%，暖态中位 ~250-700ms，输出 laya_results.json。

✓ **预期结果：**末尾打印四行准确率 + 延迟统计；中文题会看到 urgency 大量判 2 的“异常”，这是下一步的伏笔。

第 4 步：中文 criteria 对照实验 (lab02b_zh_criteria.py) ——本指南最大彩蛋

怎么操作：把 12 道中文题的 criteria/instructions 全部换成中文再跑（脚本内已写好 ZH_QUESTIONS）：

```
.venv\Scripts\python.exe exercise\lab02b_zh_criteria.py
```

目的：验证"urgency 饱和在 2"是措辞错配而非模型坏。**实现了什么：**urgency 4/12→7/12、dept 8/12→10/12，分数分布不再饱和。

✅ **预期结果：**dept 10/12 urgency 7/12 churn 10/12。改一个措辞涨 3 题这就是"决策模型的提示词工程"。

第 5 步：云端大模型同卷对比 (lab03_llm_benchmark.py)

怎么操作：

```
.venv\Scripts\python.exe exercise\lab03_llm_benchmark.py
```

目的：同一张 24 题考卷喂给 claude CLI（本机配置 kimi-for-coding 供应商），量出"云端 vs 本地"两条腿。**实现了什么：**95.8% / 83.3% / **100%**，单题中位 6,693ms。

✅ **预期结果：**准确率各维度不低于 Laya（大模型判断力强），但单题耗时是 Laya 的几十倍；输出 llm_results.json。

第 6 步（选修）：给 Laya 当裁判接进 jev-router

思路：jev-router 的 SDK 有 TYPESAFE_BASE_URL 接缝（[[接缝与桩实现StubSeam]]），laya-server / 1Panel-dev/laya-server 提供 TypeSafe 兼容 API——把接缝指向本地 laya-server，整条"逐轮模型路由"就完全本地化了。本项目未展开（需另起一篇），列为下一步。

🌟 这个项目给我什么（能力清单）

1. **会装会跑：**pip 一行装上，CPU 就能跑 421M 决策模型，暖态 ~250ms/题，GPU 可到 33ms。
2. **会出考卷：**自建带标准答案的分诊测试集 + 三维度准确率/延迟统计的评测脚本，可复用到任何"本地 vs 云端"对比。
3. **会调措辞：**criteria 是决策模型的 prompt——语言错配会造成"全打 2"式的静默劣化，换语言措辞立竿见影。
4. **会读评测：**零样本/微调/自建考卷/公开基准四个口径不再打架，厂商自宣会打折。
5. **会挑部署路线：**Python/ONNX/MLX/laya.cpp/HTTP server，按平台和延迟预算选。
6. **会接生产：**置信度门控、弃权阈值拟合、温度校准工具链（fit_abstention_thresholds 等）。

产出：F:\reminder\laya-decision-model\ · 拾遗任务 1a45286b (<https://x.com/xiaomovps/status/2101111530313933182>) · 知识入库：
vault/概念/非自回归决策模型 等 · 2026-10-03